AIMERZ.ai
Aim.Act.Achieve

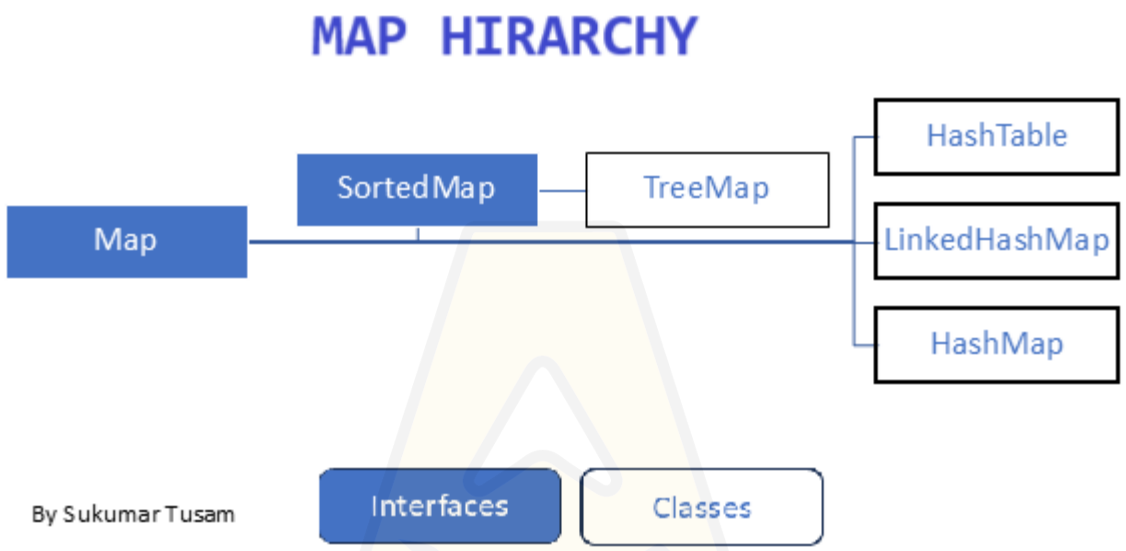
NOTES

Collections Framework/API - III

MAP & Properties Class

Map Interface:

The **Map interface** in Java is part of the **java.util** package and is used to represent a collection of key-value pairs. Unlike other collection types like List or Set, a **Map** is not part of the Collection interface hierarchy. It provides a way to store, retrieve, and manipulate data based on unique keys.



A Map represents a key-value pair structure where each key is associated with exactly one value. Duplicate keys are not allowed.

Features of the Map Interface

Key-Value Pair Storage:

- ➔ Each element is stored as a key-value pair (key, value).
- ➔ Keys are unique, but values can be duplicated.

No Direct Iteration Over Keys and Values:

➔ Unlike List or Set, you cannot iterate directly. Instead, keys, values, or entries must be accessed through dedicated views (keySet, values, or entrySet).

Non-Indexed Collection:

➔ Unlike List, elements in a Map are not stored in an indexed sequence. They are accessed by keys.

Efficient Lookups:

➔ Provides fast access to values using keys, often with constant-time complexity depending on the implementation.

Key Implementations: The Map interface is extended by various classes and interfaces:

Interface/Class	Description
HashMap	A hash table-based implementation. Allows null keys and values. Does not maintain any order.
LinkedHashMap	Extends HashMap. Maintains insertion order.
TreeMap	A sorted map based on the red-black tree. Maintains natural or custom order of keys.
Hashtable	Synchronized version of HashMap. Does not allow null keys or values.
ConcurrentHashMap	A thread-safe implementation with better performance than Hashtable.
SortedMap	An interface that extends Map to provide sorting functionality.
NavigableMap	Extends SortedMap with methods for navigation like retrieving keys higher, lower, or equal to a given key.

Let's see some, **Common Methods of Map Interface**

Method	Description
put(K key, V value)	Associate the specified value with the specified key.
get(Object key)	Returns the value associated with the specified key.
remove(Object key)	Removes the entry for the specified key.
containsKey(Object key)	Checks if the map contains the specified key.
containsValue(Object value)	Checks if the map contains the specified value.
keySet()	Returns a Set of all keys.
values()	Returns a Collection of all values.
entrySet()	Returns a Set of key-value pairs (Map.Entry objects).
size()	Returns the number of key-value mappings.
clear()	Removes all entries from the map.
isEmpty()	Checks if the map is empty.
replace(K key, V value)	Replaces the value associated with the specified key, if it exists.

Real-World Analogy: Locker System

Think of a **Map** as a **locker system** at a gym:

➔ **Keys**: The locker numbers (unique).

- **Values**: The items stored in the lockers.
- You can quickly find your belongings by knowing the locker number (key).
- Two lockers can hold identical items (duplicate values), but each locker number (key) is unique.

Let's begin with, **HashMap in Java**

The **HashMap** is a widely used class in Java's java.util package that implements the Map interface. It is a collection designed for efficient storage and retrieval of key-value pairs using a hashing mechanism.

Now let's see the **Key Features of HashMap**

Key-Value Pair Storage:

- Stores data as a combination of keys and values (key, value).
- Keys are unique, while values can be duplicated.

Null Keys and Values:

- Allows one null key and multiple null values.

Unordered:

- Does not guarantee the order of elements (keys or values).

Hashing Mechanism:

- Uses a hash table to store elements, enabling fast access using the key's hash code.

Non-Synchronized:

- Not thread-safe by default; requires explicit synchronization for concurrent access.

Efficient Performance:

➔ Average time complexity for basic operations like get() and put() is **O(1)**.

Resizeable:

➔ Automatically resizes when the number of elements exceeds the load factor (default is 0.75).

HashMap Hierarchy

The HashMap class is part of the Map interface hierarchy:

```
Java
java.lang.Object
  ↳ java.util.AbstractMap<K,V>
      ↳ java.util.HashMap<K,V>
```

HashMap Code Example:

```
Java
import java.util.HashMap;
public class HashMapExample {
    public static void main(String[] args) {
        // Create a HashMap of String-Integer pairs
        HashMap<String, Integer> scores = new HashMap<>();

        // Add key-value pairs to the HashMap
        scores.put("John", 85);
```

```
        scores.put("Alice", 92);
        scores.put("Bob", 75);

        // Retrieve a value by its key
        System.out.println("John's score: " + scores.get("John"));

        // Check if a key exists
        System.out.println("Contains Alice? " +
scores.containsKey("Alice"));

        // Remove an entry by key
        scores.remove("Bob");

        // Iterate through the keys and values
        for (String key : scores.keySet()) {
            System.out.println(key + ": " + scores.get(key));
        }

        // Check size
        System.out.println("Size of HashMap: " + scores.size());
    }
}
```

Output:

```
Java
John's score: 85
Contains Alice? true
Alice: 92
```

```
John: 85
Size of HashMap: 2
```

Constructors of HashMap

Constructor	Description
HashMap()	Creates an empty HashMap with default initial capacity (16) and load factor (0.75).
HashMap(int initialCapacity)	Creates a HashMap with the specified initial capacity.
HashMap(int initialCapacity, float loadFactor)	Creates a HashMap with the specified initial capacity and load factor.
HashMap(Map<? extends K,? extends V> m)	Creates a HashMap with the same mappings as the specified Map.

Handling Null Keys and Values in HashMap

```Java
import java.util.HashMap;

public class NullKeyValueExample {
    public static void main(String[] args) {
        HashMap<String, String> map = new HashMap<>();
```

```
        map.put(null, "Null Key");
        map.put("Key1", null);
        map.put("Key2", "Value2");

        System.out.println(map); // Output: {null=Null Key,
Key1=null, Key2=Value2}
    }
}
```

Output:

```Java
{null=Null Key, Key2=Value2, Key1=null}
```

LinkedHashMap in Java

The **LinkedHashMap** is a class in Java's java.util package that extends HashMap and implements the Map interface. It maintains the insertion order of key-value pairs, making it ideal when you need a combination of fast access and predictable order.

Now let's see the, **Key Features of LinkedHashMap**

Order Preservation:

- ➔ Maintains the order in which elements are inserted.
- ➔ Can also maintain access order if explicitly specified (via a special constructor).

Key-Value Pair Storage:

- ➔ Stores data in key-value pairs, similar to HashMap.

Null Support:

➔ Allows one null key and multiple null values.

Efficient Retrieval:

➔ Offers constant-time performance (O(1)) for basic operations like get() and put().

Underlying Data Structure:

➔ Combines a hash table with a doubly-linked list to maintain insertion or access order.

Non-Synchronized:

➔ Not thread-safe; requires external synchronization for concurrent use.

LinkedHashMap Hierarchy

The LinkedHashMap class belongs to the Map interface hierarchy:

```Java
java.lang.Object
  ↳ java.util.AbstractMap<K,V>
      ↳ java.util.HashMap<K,V>
          ↳ java.util.LinkedHashMap<K,V>
```

Common Methods in LinkedHashMap

Method	Description
put(K key, V value)	Adds or updates a key-value pair in the map.
get(Object key)	Retrieves the value associated with the specified key.
remove(Object key)	Removes the mapping for the specified key.

keySet()	Returns a Set of all keys.
values()	Returns a Collection of all values.
entrySet()	Returns a Set of all key-value pairs.
clear()	Removes all entries from the map.
containsKey(Object key)	Checks if the map contains the specified key.
containsValue(Object value)	Checks if the map contains the specified value.
size()	Returns the number of key-value pairs.

When to use LinkedHashMap: When insertion order is important.

LinkedHashMap Code Syntax

```Java
import java.util.LinkedHashMap;

public class LinkedHashMapExample {
    public static void main(String[] args) {
        // Create a LinkedHashMap
        LinkedHashMap<Integer, String> map = new LinkedHashMap<>();

        // Add key-value pairs
        map.put(1, "Apple");
        map.put(2, "Banana");
        map.put(3, "Cherry");

        // Display map
        System.out.println("LinkedHashMap: " + map);

        // Access elements
        System.out.println("Value for key 2: " + map.get(2)); //
Output: Banana
    }
}
```

Output:

```
Java
LinkedHashMap: {1=Apple, 2=Banana, 3=Cherry}
Value for key 2: Banana
```

TreeMap in Java

The **TreeMap** is a class in Java's java.util package that implements the Map interface. It uses a **Red-Black Tree** (a self-balancing binary search tree) to store and manage key-value pairs, maintaining the **natural order** of its keys or an order defined by a provided comparator.

Now let's the, **Key Features of TreeMap**

Sorted Order:

- ➔ Automatically sorts keys in their **natural order** (e.g., numeric or lexicographical).
- ➔ Can also use a **custom comparator** for sorting.

Null Key Restriction:

- ➔ Does **not allow null keys**, as it uses comparisons for sorting.
- ➔ Allows multiple null values.

Efficient Operations:

- ➔ Provides logarithmic time complexity (O(log n)) for basic operations like put(), get(), remove().

NavigableMap Implementation:

- ➔ Extends AbstractMap and implements NavigableMap for additional navigation methods (e.g., floorKey(), ceilingKey()).

Immutable Ordering:

➔ Order of elements remains constant unless explicitly modified.

TreeMap Hierarchy

```Java
java.lang.Object
  ↳ java.util.AbstractMap<K, V>
      ↳ java.util.TreeMap<K, V>
```

Common Methods in TreeMap

Method	Description
put(K key, V value)	Adds or updates a key-value pair.
get(Object key)	Retrieves the value associated with the specified key.
remove(Object key)	Removes the mapping for the specified key.
firstKey()	Returns the smallest key in the map.
lastKey()	Returns the largest key in the map.
floorKey(K key)	Returns the largest key ≤ the given key.
ceilingKey(K key)	Returns the smallest key ≥ the given key.
subMap(K fromKey, K toKey)	Returns a view of the map for keys within a specific range.
keySet()	Returns a Set of all keys in sorted order.
values()	Returns a Collection of all values.

TreeMap Code Example

```Java
import java.util.TreeMap;

public class TreeMapExample {
    public static void main(String[] args) {
        // Create a TreeMap
        TreeMap<Integer, String> map = new TreeMap<>();

        // Add key-value pairs
        map.put(3, "Apple");
        map.put(1, "Banana");
        map.put(2, "Cherry");

        // Display the map (sorted by keys)
        System.out.println("TreeMap: " + map);

        // Access specific keys
        System.out.println("First Key: " +
map.firstKey());
        System.out.println("Last Key: " + map.lastKey());
    }
}
```

Output:

```Java
TreeMap: {1=Banana, 2=Cherry, 3=Apple}
First Key: 1
Last Key: 3
```

HashTable in Java

The **Hashtable** is a legacy class in Java that implements the Map interface. It is part of the java.util package and provides a way to store key-value pairs. The Hashtable is synchronized, meaning it is thread-safe for concurrent access but slower compared to newer classes like HashMap.

Key Features of Hashtable

Thread-Safe:

- ➔ All methods in Hashtable are synchronized, making it safe for multithreaded environments.

No Null Keys or Values:

- ➔ Unlike HashMap, Hashtable does **not allow null keys or null values**.

Hashing Mechanism:

- ➔ Internally uses a **hashing algorithm** to store and retrieve key-value pairs efficiently.

Insertion Order Not Maintained:

- ➔ The order of elements is not preserved; entries are stored based on their hash values.

Legacy Class:

- ➔ Introduced in Java 1.0 and was later retrofitted to implement the Map interface in Java 2.

Collision Handling:

- ➔ Uses **separate chaining** to resolve hash collisions.

Backward Compatibility:

➔ It remains in the JDK for compatibility with older code but is rarely used in new projects.

Hashtable Hierarchy

```Java
java.lang.Object
  ↳ java.util.Dictionary<K, V>
      ↳ java.util.Hashtable<K, V>
          ↳ java.util.Properties
```

Common Methods of Hashtable

Method	Description
put(K key, V value)	Inserts a key-value pair into the hashtable.
get(Object key)	Retrieves the value associated with the specified key.
remove(Object key)	Removes the mapping for the specified key.
containsKey(Object key)	Checks if a specific key exists.
containsValue(Object value)	Checks if a specific value exists.

size()	Returns the number of key-value pairs in the hashtable.
isEmpty()	Checks if the hashtable is empty.
keys()	Returns an Enumeration of all keys.
elements()	Returns an Enumeration of all values.

Hashtable Code Example

Java

```java
import java.util.Hashtable;

public class HashtableExample {
    public static void main(String[] args) {
        // Create a Hashtable
        Hashtable<Integer, String> hashtable = new
Hashtable<>();

        // Add key-value pairs
        hashtable.put(1, "Apple");
        hashtable.put(2, "Banana");
        hashtable.put(3, "Cherry");

        // Access elements
        System.out.println("Value for key 2: " +
hashtable.get(2));

        // Check if a key exists
        System.out.println("Contains key 3? " +
hashtable.containsKey(3));

        // Remove an element
        hashtable.remove(1);
```

```
        System.out.println("After removal: " +
hashtable);
    }
}
```

Output:

```
Value for key 2: Banana
Contains key 3? true
After removal: {3=Cherry, 2=Banana}
```

Real-World Use Case: Student Records

```
import java.util.Hashtable;

public class StudentRecords {
    public static void main(String[] args) {
        // Create a Hashtable to store student IDs and names
        Hashtable<Integer, String> students = new
Hashtable<>();

        // Add student records
        students.put(101, "Deepak");
        students.put(102, "Melvin");
        students.put(103, "Yuvraj");

        // Retrieve a student's name by ID
        System.out.println("Student with ID 102: " +
students.get(102));

        // Check if a student exists
```

```
        System.out.println("Contains ID 101? " +
students.containsKey(101));

        // Display all student records
        System.out.println("All Student Records: " +
students);
    }
}
```

Output

```Java
Student with ID 102: Melvin
Contains ID 101? true
All Student Records: {103=Yuvraj, 102=Melvin, 101=Deepak}
```

Properties Class in Java

The **Properties** class in Java is a part of the **java.util** package. It is a subclass of **Hashtable** and is used to store **key-value pairs**, where both the keys and values are String objects. It is commonly used for managing **configuration settings** or application data, such as database connection details or file paths.

Features of the Properties Class

- ➔ **Key-Value Pairs (Both as Strings)**:
  Both keys and values must be String objects, making it simple to store and retrieve data.
- ➔ **Default Settings**:
  The Properties class allows specifying default values, which are returned when no explicit value is found for a given key.

- **File I/O Support**:
  It provides built-in methods to read/write data to/from files using .properties format.
- **Thread-Safe**:
  Since **Properties** inherits from **Hashtable**, it is thread-safe, making it suitable for multi-threaded environments.
- **Backward Compatibility**:
  It has been part of Java since the early versions, making it compatible with older systems and applications.

A Real-World Analogy

Imagine you have a **recipe book** where each recipe has a unique **name (key)**, and the corresponding **instructions (value)**.

- You look up the recipe by its name to get the instructions.
- If you don't find a recipe, you refer to the **default recipe** (default settings).
- The recipe book can be updated (add/remove recipes) and shared across a kitchen staff (thread-safe).

Common Methods in the Properties Class

Here are the commonly used methods with examples:

Method	Description	Example
setProperty(key, value)	Adds a new key-value pair or updates an existing key.	properties.setProperty("username", "admin");
getProperty(key)	Retrieves the value for a given key.	String user = properties.getProperty("username");

getProperty(key, defaultValue)	Retrieves the value, or returns the default value if the key is missing.	String user = properties.getProperty("username", "guest");
load(InputStream)	Reads properties from a file.	properties.load(new FileInputStream("config.properties"));
store(OutputStream, comment)	Saves properties to a file with comments.	properties.store(new FileOutputStream("config.properties"), "App Config");
list(PrintStream)	Prints all key-value pairs to an output stream.	properties.list(System.out);

Creating and Using Properties

```java
import java.util.Properties;

public class PropertiesExample {
    public static void main(String[] args) {
        Properties properties = new Properties();

        // Adding key-value pairs
        properties.setProperty("username", "admin");
        properties.setProperty("password", "12345");

        // Retrieving values
        String user = properties.getProperty("username");
        String pass = properties.getProperty("password");
```

```
        // Printing values
        System.out.println("Username: " + user);
        System.out.println("Password: " + pass);
    }
}
```

Output

```
Java

Username: admin

Password: 12345
```

Code example explaining the usage of all the listed methods of the **Map** interface in Java using **HashMap**

```
Java
import java.util.HashMap;
import java.util.Map;
import java.util.Set;
import java.util.Collection;

public class MapMethodsDemo {
    public static void main(String[] args) {
        // Create a Map instance using HashMap
        Map<Integer, String> map = new HashMap<>();

        // put(K key, V value): Add key-value pairs to the map
        map.put(1, "Apple");
        map.put(2, "Banana");
        map.put(3, "Cherry");
        map.put(4, "Date");

        // Display the map
        System.out.println("Initial Map: " + map);
```

```
        // get(Object key): Retrieve the value associated with a
specific key
        System.out.println("Value for key 2: " + map.get(2)); //
Banana

        // remove(Object key): Remove a key-value pair by key
        map.remove(4);
        System.out.println("Map after removing key 4: " + map);

        // containsKey(Object key): Check if a specific key exists
        System.out.println("Contains key 3? " + map.containsKey(3));
// true

        // containsValue(Object value): Check if a specific value
exists
        System.out.println("Contains value 'Apple'? " +
map.containsValue("Apple")); // true

        // keySet(): Get all the keys as a Set
        Set<Integer> keys = map.keySet();
        System.out.println("Keys in the map: " + keys);

        // values(): Get all the values as a Collection
        Collection<String> values = map.values();
        System.out.println("Values in the map: " + values);

        // entrySet(): Get all key-value pairs as a Set of Map.Entry
        Set<Map.Entry<Integer, String>> entries = map.entrySet();
        System.out.println("Entries in the map: " + entries);

        // size(): Get the number of key-value pairs in the map
        System.out.println("Size of the map: " + map.size());

        // clear(): Remove all key-value pairs from the map
        map.clear();
        System.out.println("Map after clear(): " + map);

        // isEmpty(): Check if the map is empty
        System.out.println("Is the map empty? " + map.isEmpty()); //
true
```

```
        // replace(K key, V value): Replace the value for a specific
key
        map.put(1, "Apple");
        map.put(2, "Banana");
        map.replace(2, "Blueberry");
        System.out.println("Map after replacing value for key 2: " +
map);
    }
}
```

Output

```Java
Initial Map: {1=Apple, 2=Banana, 3=Cherry, 4=Date}
Value for key 2: Banana
Map after removing key 4: {1=Apple, 2=Banana, 3=Cherry}
Contains key 3? true
Contains value 'Apple'? true
Keys in the map: [1, 2, 3]
Values in the map: [Apple, Banana, Cherry]
Entries in the map: [1=Apple, 2=Banana, 3=Cherry]
Size of the map: 3
Map after clear(): {}
Is the map empty? true
Map after replacing value for key 2: {1=Apple, 2=Blueberry}
```

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan